

Analyse de securite (MSPR3 / TPRE601)

Analyse de securite de la plateforme MSPR HealthAI Coach mise en production localement via Docker Compose. Elle confronte l'existant de la plateforme a trois referentiels : OWASP Top 10 (2021), RGPD et NIST Cybersecurity Framework. Le perimetre couvre les 8 services applicatifs, l'orchestration Compose, les scripts d'exploitation et la chaine CI/CD. L'application mobile, client de la plateforme pris en charge par un autre membre de l'equipe, est decrite dans `application-mobile.md` : elle n'expose pas de service cote serveur et reste hors du perimetre de cette analyse.

Le document reste factuel : il ne decrit que les controles reellement presents dans le depot et signale explicitement ce qui n'est pas couvert.

1. OWASP Top 10 (2021)

Categorie	Etat	Elements en place dans la plateforme
A01 Broken Access Control	Partiel	Authentification deleguee a better-auth (email/mot de passe + JWT). Les services metier (API Spring, AI-Nutrition, Reco-Fitness) valident le JWT via le secret partage <code>BETTER_AUTH_SECRET</code> . Endpoints proteges cote API.
A02 Cryptographic Failures	Partiel	JWT signes en HS512. Cles API (Resend, Mistral) chiffrees au repos via <code>openssl aes-256-cbc (secrets/*.enc , PBKDF2 100000 iterations).</code> <code>.env</code> exclu du versionnement. Mots de passe des bases generes par <code>bootstrap.sh</code> .
A03 Injection	Partiel	API Spring sur JPA/requetes parametrees. Validation des entrees cote services (Spring + FastAPI).
A04 Insecure Design	Partiel	Segmentation reseau (Ollama isole sur <code>ai_internal</code>), separation des bases (metier / auth), delegation de l'authentification a un service dedie.
A05 Security Misconfiguration	Partiel	Conteneurs en utilisateur non-root, <code>no-new-privileges:true</code> sur tous les services, ports des bases (5433/5434/27018) bindes sur <code>127.0.0.1</code> , rotation des logs json-file. Plus de mot de passe <code>password</code> par default en exploitation.
A06 Vulnerable and Outdated Components	Partiel	Images applicatives publiees par la CI, deploiement par tags GHCR. Aucun scan de vulnerabilite d'images (voir limites).
A07 Identification and Authentication Failures	Partiel	Gestion des comptes, sessions et JWT assuree par better-auth. Verification email via Resend (desactivable en mode offline).
A08 Software and Data Integrity Failures	Partiel	Images tracables par SHA de commit (tag <code>type=sha</code>), publication uniquement si lint et tests passent.
A09 Security Logging and Monitoring Failures	Partiel	Stack d'observabilite (overlay monitoring) : Prometheus, Grafana, Loki, Promtail, Alertmanager + exporters. Logs de tous les conteneurs centralises dans Loki. 3 alertes Prometheus.
A10 Server-Side Request Forgery (SSRF)	Non evalue	Pas de controle specifique identifie dans le depot.

Detail des controles presents

A01 - Controle d'accès. L'authentification est entierement deleguee au service `mspr-auth` (better-auth). Les services consommateurs partagent le secret HMAC `BETTER_AUTH_SECRET` (injecte par variable d'environnement) et valident la signature du JWT a chaque requete. L'API expose toutefois `/api/actuator/prometheus` sans JWT pour permettre le scraping interne par Prometheus ; les endpoints `/metrics` des FastAPI et d'Auth sont egalement ouverts sur le reseau Docker. C'est documente et assume comme acceptable en environnement de demonstration.

A02 - Cryptographie et secrets. Les JWT sont signes en HS512 (secret de 64 octets minimum, genere par `openssl rand -base64 64`). Les cles API tierces (Resend, Mistral) ne sont jamais stockees en clair dans le depot : elles sont chiffrees dans `secrets/resend.enc` et `secrets/mistral.enc` (`aes-256-cbc` , sel + PBKDF2, 100000 iterations) et dechiffrees au demarrage par `bootstrap.sh` a partir d'une passphrase. Le `.env` est exclu par `.gitignore` , de meme que `backups/` et `.passphrase.local` .

A05 - Configuration. Le durcissement repose sur des reglages communs Compose (`x-hardening`) appliques a tous les services : `no-new-privileges:true` et rotation des logs (`max-size 10m` , `max-file 3`). Les 6 services applicatifs tournent en utilisateur non-root (defini dans leurs Dockerfile respectifs) et exposent un `HEALTHCHECK` . Les ports des trois bases de donnees sont bindes sur `127.0.0.1` , donc non exposes sur le reseau de l'hote. Les mots de passe des bases (`DB_PASSWORD` , `AUTH_DB_PASSWORD`) sont generes aleatoirement par `bootstrap.sh` s'ils sont absents, evitant le default `password` en exploitation.

A09 - Journalisation et supervision. Promtail collecte la sortie standard et d'erreur de tous les conteneurs vers Loki (retention 7 jours), interrogeable dans Grafana. Les exporters (cAdvisor, node-exporter, postgres-exporter x2, mongodb-exporter) et l'instrumentation applicative (Actuator/Micrometer cote API, prometheus-fastapi-instrumentator cote FastAPI, @hono/prometheus cote Auth) alimentent Prometheus. Trois regles d'alerte sont definies : `CibleInjoignable` (`up == 0`), `ConteneurCpuEleve` et `ConteneurMemoireElevee` .

Limites assumees (OWASP)

- **Pas de scan de vulnerabilite d'images** (type Trivy/Grype) ni d'analyse de dependances automatisee dediee a la securite. Le tag `:latest` est utilise pour le deploiement prod par default.
- **Endpoints de metriques non authentifies** (Actuator/Prometheus de l'API, `/metrics` des FastAPI et d'Auth), acceptable uniquement en contexte de demonstration locale.
- **Grafana en `admin/admin`** par default (surchargeable via `GRAFANA_ADMIN_PASSWORD`).
- **Pas de TLS** sur les services exposes (HTTP en local) ; pas de WAF, de gestionnaire de secrets externe ni de service mesh.
- A03 (injection) et A10 (SSRF) ne font pas l'objet de controles dedies explicites au-dela de l'usage de JPA et de la validation d'entrees.

2. RGPD

Anonymisation et minimisation des donnees

Les datasets sources charges par l'ETL dans la base metier `healthai` sont anonymises : ils ne contiennent ni donnee a caractere personnel identifiante, ni identifiant commun permettant de relier un enregistrement a une personne. La table `users` a ete **supprimee** de la base metier (migration `v7__drop_users_table.sql`),

avec retrait des cles etrangeres `user_id` dans `nutrition_entries`, `exercise_entries` et `biometric_entries`. La base metier ne stocke donc aucune PII.

Cela respecte le principe de **minimisation** (article 5 RGPD) : seules les donnees necessaires aux traitements analytiques (exercices, nutrition, biometrie agregee, recommandations) sont conservees, sans rattachement nominatif.

Donnees de sante

Les donnees manipulees (biometrie, nutrition, recommandations fitness) relevent du domaine de la sante. Du fait de l'anonymisation decrite ci-dessus, les enregistrements charges ne sont pas rattaches a des individus identifiabiles dans la base metier.

Gestion des comptes utilisateurs

Les comptes utilisateurs (email, mot de passe, sessions) sont geres exclusivement par le service `mspr-auth` (better-auth) dans une **base de donnees separee** (`auth_db`, port 5433 binde sur `127.0.0.1`). Cette separation isole les seules donnees personnelles (emails de comptes) du reste de la plateforme. Les mots de passe sont geres par better-auth (stockage hashe cote `account.password`).

Limites assumees (RGPD)

- Le cycle de vie des comptes (suppression, export, droit a l'oubli) repose sur les fonctionnalites natives de better-auth ; aucun workflow RGPD specifique (anonymisation a la demande, purge automatique) n'est implemente dans le depot.
- Pas de chiffrement au repos des bases de donnees au-dela de l'isolement reseau.

3. NIST Cybersecurity Framework

Mapping des elements existants sur les cinq fonctions du NIST CSF.

Fonction	Couverture	Elements presents
Identify	Partiel	Inventaire des 8 services et de leurs depots (README, CLAUDE.md). Cartographie reseau (<code>mspr_data_network</code> , <code>ai_internal</code>). Documentation des donnees collectees (monitoring/README.md).
Protect	Oui	Authentification/JWT (better-auth + secret partage), durcissement conteneurs (non-root, <code>no-new-privileges</code> , healthchecks), isolement reseau et binding <code>127.0.0.1</code> des bases, secrets chiffres et generes, CORS configure, rate limiting (slowapi) sur les FastAPI, validation des entrees.
Detect	Oui	Stack d'observabilite (Prometheus, Grafana, Loki, Promtail, Alertmanager + exporters). Metriques infra et applicatives. 3 alertes (cible injoignable, CPU eleve, memoire elevee).
Respond	Partiel	Alertmanager recoit et regroupe les alertes. Pas de procedure de reponse formalisee ni de routage de notification configure.
Recover	Oui	Scripts <code>scripts/backup.sh</code> (pg_dump base metier + base auth + mongodump), <code>scripts/restore.sh</code> (restauration de la derniere sauvegarde ou d'une sauvegarde precise) et <code>scripts/clean.sh</code> (remise a zero). Politique <code>restart: unless-stopped</code> sur les services.

Detail par fonction

Protect. C'est la fonction la mieux couverte : combinaison de l'authentification deleguee, du durcissement des conteneurs (reglages communs `x-hardening` appliques a tous les services), de la segmentation reseau (Ollama joignable uniquement via `ai_internal`), de la gestion des secrets (chiffrement au repos + generation au demarrage) et des controles applicatifs (CORS, rate limiting cote FastAPI, validation d'entrees).

Detect. L'overlay de monitoring fournit une supervision metriques + logs sur l'ensemble de la stack. La detection se limite a trois alertes basiques (disponibilite et ressources) ; il n'y a pas de detection d'intrusion ni d'analyse de comportement.

Recover. Les trois scripts d'exploitation couvrent la sauvegarde et la restauration des trois bases (deux PostgreSQL + MongoDB) ainsi que la remise a zero. Les sauvegardes sont locales et horodatees (`backups/<timestamp>/`, non versionnees). Il n'existe pas de sauvegarde externalisee ni de strategie de retention automatisee.

Limites assumees (NIST)

- **Respond** : pas de plan de reponse a incident documente, pas de canal de notification configure dans Alertmanager (les alertes sont regroupees mais non routees).
- **Recover** : sauvegardes locales uniquement, pas d'externalisation ni de test de restauration automatise.

4. Conclusion

Points forts

- Authentification centralisee et deleguee (better-auth, JWT HS512), validee par les services consommateurs via un secret partage.
- Durcissement des conteneurs homogene : non-root, `no-new-privileges`, healthchecks, bases non exposees hors de l'hote (`127.0.0.1`), rotation des logs.
- Gestion des secrets : cles API chiffrees au repos (aes-256-cbc + PBKDF2), mots de passe des bases generes a l'amorcade, `.env` hors versionnement.
- RGPD : anonymisation des datasets, suppression de la table `users` (V7), isolement des donnees de comptes dans une base auth dediee.
- Observabilite complete (metriques + logs centralises) couvrant la fonction Detect du NIST CSF.
- CI/CD par depot avec tests, lint, seuils de couverture (80 % API et Reco-Fitness), analyse SonarCloud cote API et publication d'images tracables (SHA de commit).

Pistes d'amelioration realistes

- Ajouter un **scan de vulnerabilite d'images** (Trivy/Grype) a la CI et figer le tag de deploiement plutot que `:latest`.
- **Authentifier les endpoints de metriques** ou les restreindre au reseau de supervision, et **changer le mot de passe Grafana** par default.
- Mettre en place **TLS** sur les services exposes pour un deploiement hors poste local.
- Formaliser un **plan de reponse a incident** et configurer un canal de notification Alertmanager (fonction Respond).

- **Externaliser les sauvegardes** et automatiser un test de restauration.
- Implementer un **workflow RGPD** explicite cote comptes (export, suppression sur demande).